Course: WiDS: Text Sentiment Analysis API

Student Name: Neena S Nair

Roll Number: 24M0153

Date: January 2026

Assigment 1: Fundamentals of Data Analysis with Python

1.1 Executive Summary

This section of the coursework focused on establishing a robust computational foundation for Data Science. The primary objective was to transition from general-purpose programming to high-performance data manipulation using Python's scientific stack. The assignment covered data structure optimization using **Lists and Dictionaries**, numerical computation with **NumPy**, and structured data analysis using **Pandas**. These tools form the prerequisite infrastructure for the advanced machine learning tasks explored in Part 2 of this report.

1.2 Python Data Structures for Data Science

Before implementing external libraries, it is crucial to understand Python's native data handling capabilities. The assignment explored the distinctions between ordered sequences and key-value mappings.

- **Lists vs. Dictionaries:**
    - **Lists** were utilized for ordered collections where element position is critical. This is foundational for time-series data or sequential processing.
    - **Dictionaries** were implemented for fast lookups using unique keys, simulating database-like retrieval where access speed is prioritized over order.
- **Modular Programming:**
    - To maintain code scalability, I implemented modular functions for repetitive mathematical operations. This practice is essential in data science pipelines to ensure that data preprocessing steps (like normalization or cleaning) are reusable and consistent across different datasets.

1.3 High-Performance Computing with NumPy

Standard Python lists are computationally expensive for large-scale mathematical operations. This module introduced **NumPy (Numerical Python)**, which provides support for large, multi-dimensional arrays and matrices.

1.3.1 Array Initialization

I explored various methods to initialize data structures, which is a critical step in setting up parameters for machine learning models:

- **Manual Construction:** Creating arrays from scratch to understand structure.
- **Automated Generation:** Using functions to generate arrays of zeros or random numbers. This is directly applicable to initializing weight matrices in neural networks, where starting with random weights is standard practice.

1.3.2 Slicing and Dimensionality

Data often comes in large blocks, and specific features need to be extracted for analysis.

- **Slicing:** I practiced advanced slicing techniques on 1D and 2D arrays (e.g., arr[:3, 1:]). This allows for the efficient extraction of specific rows (samples) and columns (features) without copying the underlying data, optimizing memory usage.
- **Shape Analysis:** Utilizing the .shape attribute was emphasized as a debugging tool. In matrix multiplication (a core operation in Deep Learning), ensuring that the dimensions of input tensors align is the most common source of error.

1.4 Structured Data Analysis with Pandas

While NumPy handles numerical data, real-world data is often tabular and mixed-type (text, numbers, dates). **Pandas** was used to bridge this gap.

- **DataFrame Construction:** I created DataFrames from raw lists and dictionaries, effectively converting unstructured code outputs into structured tables ready for analysis.
- **Advanced Indexing (loc vs iloc):** A key learning outcome was distinguishing between label-based and integer-based indexing:
    - loc: Used for accessing data by human-readable labels (e.g., finding a row by a specific date or name).
    - iloc: Used for accessing data by integer position (e.g., "get the 5th row"), which is crucial when iterating through datasets programmatically.
- **Conditional Filtering:** I implemented boolean indexing to filter data based on specific conditions. For example, retrieving performance statistics for a specific player without

manually scanning the dataset. This simulates the "querying" capability of SQL directly within Python.

Assignment 2: Comparative Analysis of Classical vs. Neural NLP

2.1 Project Overview

Following the foundational work in Python, the second phase of the coursework focused on **Natural Language Processing (NLP)**. The objective was to compare a classical statistical approach against a modern neural network architecture. The study used the **COVID-19 NLP Text Classification** dataset to determine if the computational complexity of Deep Learning yields significant improvements in fine-grained sentiment analysis.

2.2 Theoretical Framework: Tokenization and Embeddings

A core challenge in NLP is converting raw text into a numerical format that computers can process. The report investigated two critical mechanisms: **Tokenization** and **Embeddings**.

2.2.1 The Necessity of Tokenization

Computers cannot "read" text; they process numbers. Tokenization acts as the translator, chopping text into smaller units (tokens) and converting them into ID numbers. It is the bridge between human language and machine logic.

- **Impact of Poor Tokenization:**
    - **The "Unknown Word" Issue:** Rigid tokenizers fail to recognize rare words, replacing them with <UNK> tags and losing critical information.
    - **Breaking Meaning:** Illogical splitting (e.g., breaking "unhappiness" into "u-nh-app") prevents the model from understanding root words, degrading accuracy.

2.2.2 The Role of Word Embeddings

Simply assigning random IDs to words (e.g., "Dog"=4, "Cat"=99) is insufficient because it captures no relationship between them. **Embeddings** solve this by converting IDs into dense vectors. In this vector space, synonyms are mathematically close, allowing the model to understand context—for example, knowing that "King" is related to "Queen" or that "Catastrophe" is semantically stronger than "Bad".

2.3 Methodology

Two distinct systems were developed to classify the sentiment of COVID-19 tweets.

System A: Classical Statistical Approach (Baseline)

- **Technique:** TF-IDF Vectorization (Term Frequency-Inverse Document Frequency).
- **Model:** Logistic Regression.
- **Logic:** This model operates on a "Bag of Words" assumption. It assigns sentiment based purely on the frequency of specific keywords (e.g., "panic", "crisis") without considering word order or context. It builds a simple linear decision boundary based on the presence of positive or negative words.

System B: Neural Approach (Deep Learning)

- **Technique:** Learned Word Embeddings.
- **Model:** Keras Sequential Network.
- **Architecture:**
    1. **Embedding Layer:** Maps words to dense vectors (60 dimensions).
    2. **Global Average Pooling:** Condenses the vector information.
    3. **Dense Layers:** Fully connected layers to classify the sentiment[21].
- **Logic:** Unlike System A, this model captures the *intensity* and *context* of words. It learns that "crisis" is not just a negative word, but a severe one, and it considers the semantic relationship between words in the sentence.

2.4 Experimental Results

The models were evaluated on the same test set, yielding the following performance metrics:

Feature	System A (Classical)	System B (Neural)
Technique	TF-IDF + Logistic Regression	Embeddings + Neural Network
Accuracy	**56.40%**	**66.22%**
Training Time	< 5 Seconds	~30 Seconds
Strengths	Extremely fast training; easy to interpret.	Higher accuracy; captures context.
Weaknesses	Ignores word order; struggles with nuance.	Requires more computational power.

2.5 Analysis and Observations

2.5.1 The Performance Gap

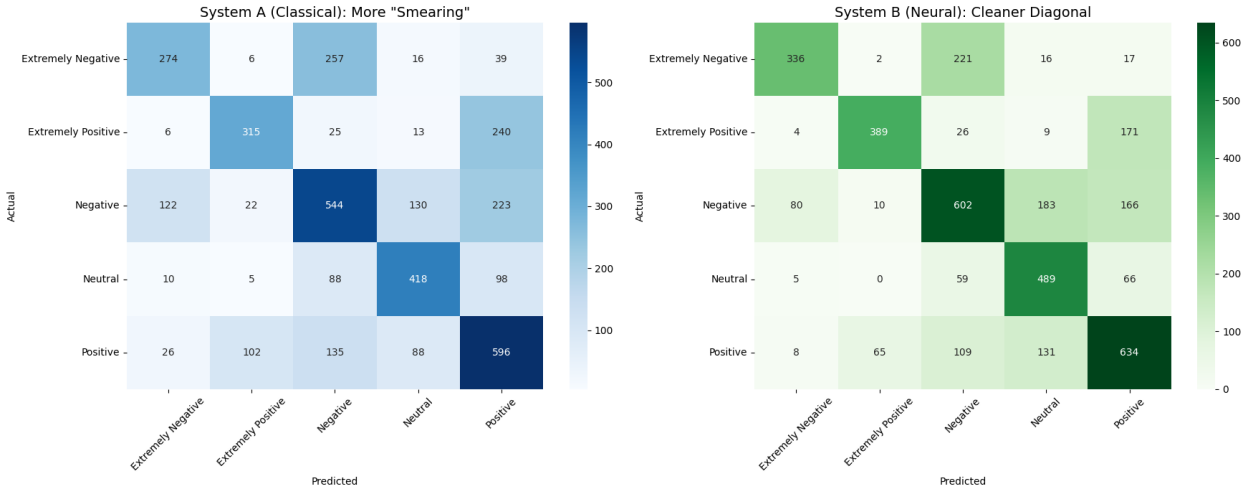
System B (Neural Network) outperformed the classical baseline by approximately **10%**. This significant gap highlights the limitations of the Bag-of-Words approach. The dataset utilized fine-grained labels (e.g., "Extremely Positive" vs. "Positive"). System A struggled to distinguish between these intensities because it treats all negative words similarly. System B, leveraging embeddings, successfully captured semantic intensity, allowing it to differentiate between a moderate inconvenience and a severe crisis.

2.5.2 Error Analysis (Confusion Matrices)

Visualizing the confusion matrices provided deeper insights into the classification errors:

- **System A (Smearing):** The classical model showed significant "smearing" across the diagonal. It frequently confused "Extremely Negative" with "Negative," indicating it could detect the *polarity* (good vs. bad) but not the *intensity*.
- **System B (Clean Diagonal):** The neural model produced a much cleaner diagonal, correctly classifying the fine-grained labels more often. This proves that the embedding layer successfully encoded the semantic weight of the vocabulary.

COMPARISON CONFUSION MATRICES BETWEEN TWO SYSTEMS



2.6 Conclusion

This two-part study demonstrates the progression of a Data Scientist's toolkit. Part 1 established the coding efficiency required to handle data, while Part 2 applied those skills to solve complex NLP problems.

While the Classical Model (System A) offers speed and simplicity, it lacks the nuance required for modern text analysis tasks. The **10% accuracy gain** achieved by the Neural Network (System B) confirms that for tasks involving sentiment intensity and semantic context, deep learning architectures are superior. The ability to map words into a continuous vector space allows neural networks to "understand" language in a way that simple frequency counting cannot.